

Documentation Technique - Module Business Intelligence (BI)

Projet : LinkedIn Agent Automation

Auteur : [Votre Nom]

Date : Mai 2026

Table des matières

- [Introduction](#)
- [Objectifs et Importance](#)
- [Architecture Technique](#)
- [Composants du Dashboard BI](#)
- [Implémentation Technique](#)
- [Base de Données et Modélisation](#)
- [Défis Techniques et Solutions](#)
- [Conclusion](#)

Introduction

Le module **Business Intelligence (BI)** est un tableau de bord analytique intégré à l'application LinkedIn Agent Automation. Il permet aux utilisateurs de visualiser, analyser et optimiser leur performance de prospection LinkedIn en temps réel.

Ce dashboard BI centralise toutes les métriques clés de la prospection automatisée et offre des insights actionnables pour améliorer les taux de conversion et l'efficacité des campagnes.

Objectifs et Importance

Objectifs Principaux

- Centralisation des données** : Agréger toutes les données de prospection (prospects, messages, conversions, actions agent) dans une vue unifiée
- Visualisation en temps réel** : Afficher les KPIs et tendances avec des filtres temporels (7, 30, 90 jours)
- Optimisation des campagnes** : Identifier les templates performants, les créneaux horaires optimaux, et les goulets d'étranglement
- Prise de décision data-driven** : Fournir des insights générés par l'IA pour guider les actions marketing

Importance pour l'Utilisateur

- Gain de temps** : Visualisation instantanée de la performance sans export Excel manuel
- Optimisation ROI** : Identification des templates et timings les plus efficaces
- Détection précoce** : Alertes sur les baisses de performance ou taux d'échec de l'agent
- Personnalisation** : Adaptation des messages selon les patterns de réponse détectés

Importance Technique

- Architecture scalable** : Séparation claire entre frontend (React/Next.js) et backend (API routes PostgreSQL)
- Performance** : Utilisation de requêtes SQL optimisées avec agrégations côté serveur
- Extensibilité** : Structure modulaire permettant d'ajouter facilement de nouveaux modules BI

Architecture Technique

Stack Technologique

Couche	Technologie	Rôle
Frontend	React + Next.js 14	Framework web avec rendu serveur

UI Components	shadcn/ui + Tailwind CSS	Bibliothèque de composants et styling
Visualisations	Recharts	Bibliothèque de graphiques (Sankey, Scatter, Treemap, etc.)
Animations	Framer Motion	Animations fluides et transitions
Backend	Next.js API Routes	Endpoints REST pour les données BI
Base de données	PostgreSQL (Neon)	Stockage des données de prospection
Client DB	pg (node-postgres)	Pool de connexions PostgreSQL

Flux de Données



Structure des Fichiers

```
linkedin-agent-frontend/
├─ app/api/bi/
│  ├─ kpi/route.ts           # KPIs principaux
│  ├─ conversion/route.ts    # Funnel, cycle time, cohortes
│  ├─ templates/route.ts     # Performance templates, heatmap
│  ├─ geo/route.ts           # Géographie, industries, ICP
│  └─ agent/route.ts         # Analytics agent IA
├─ components/bi/
│  ├─ BIShell.tsx           # Conteneur principal
│  ├─ KPIHero.tsx           # Cartes KPI animées
│  ├─ AIInsights.tsx        # Insights générés par IA
│  ├─ ConversionIntelligence.tsx # Sankey + Funnel
│  ├─ TemplateLab.tsx       # A/B testing templates
│  ├─ ProspectMap.tsx       # Carte géographique + ICP
│  ├─ AgentAnalytics.tsx    # Analytics agent
│  ├─ Forecast.tsx          # Prédictions
│  └─ biTypes.ts            # Types TypeScript partagés
└─ db/
   ├─ schema.sql            # Schéma de la base
   └─ seed-*.js             # Scripts de peuplement
```

Composants du Dashboard BI

1. BIShell - Conteneur Principal

Fichier : `components/bi/BIShell.tsx`

Rôle : Orchestrateur du dashboard BI, gère l'état global (filtre temporel) et compose tous les modules.

Fonctionnalités :

- Filtre temporel (7j, 30j, 90j) partagé entre tous les modules
- Export PDF via `window.print()`
- Header avec titre "Tableau de bord" et date en temps réel
- Layout responsive (mobile-first)

Code clé :

```
const [range, setRange] = useState<BIRange>(30);
// range est propagé à tous les composants enfants
<KPIHero range={range} />
<AIInsights range={range} />
<ConversionIntelligence range={range} />
```

2. KPIHero - Indicateurs Clés de Performance

Fichier : `components/bi/KPIHero.tsx`

Rôle : Affiche 6 KPIs principaux avec indicateurs de variation (delta).

KPIs affichés :

1. **Nouveaux prospects** - Volume de prospects identifiés
2. **Messages envoyés** - Volume de messages sortants
3. **Taux de réponse** - % de réponses / messages envoyés
4. **Conversions** - Prospects convertis en clients
5. **Actions agent IA** - Nombre d'actions automatisées
6. **Score moyen ICP** - Qualité moyenne des prospects (0-100)

Fonctionnalités :

- Calcul automatique du delta (variation vs période précédente)
- Indicateurs visuels (flèche verte/rouge) selon la tendance
- Animations d'entrée séquencées avec Framer Motion
- Skeletons de chargement

API endpoint : `/api/bi/kpi?range={jours}`

SQL utilisé :

```
-- Comparaison période courante vs précédente
WITH current AS (
  SELECT COUNT(*) as prospects, ...
  FROM prospects WHERE created_at > NOW() - INTERVAL '${range} days'
),
previous AS (
  SELECT COUNT(*) as prospects, ...
  FROM prospects WHERE created_at > NOW() - INTERVAL '${range * 2} days'
  AND created_at <= NOW() - INTERVAL '${range} days'
)
SELECT current.*, previous.*,
  (current.prospects - previous.prospects) * 100.0 / previous.prospects as delta
```

3. AIInsights - Narration Générée par IA

Fichier : `components/bi/AIInsights.tsx`

Rôle : Analyse les données de plusieurs endpoints BI pour générer des insights narratifs actionnables.

Types d'insights générés :

- **Win** : Succès (ex: taux de réponse excellent)
- **Loss** : Problème détecté (ex: taux de réponse faible)
- **Tip** : Recommandation (ex: créneau optimal identifié)
- **Info** : Information contextuelle

Logique d'analyse :

```
// Exemple : détection du meilleur créneau horaire
const best = heatmap.reduce((b, h) => {
  if (h.sent < 3) return b; // Ignorer les créneaux avec peu de données
  return !b || h.rate > b.rate ? h : b;
}, null);
if (best && best.rate > 0) {
  out.push({
    type: "tip",
    title: "Créneau optimal identifié",
    detail: `Vos messages envoyés ${DOW[best.dow]} à ${best.hour}h obtiennent ${best.rate}% de réponses.`,
    metric: `${DOW[best.dow]} ${best.hour}h`
  });
}
```

Sources de données :

- KPIs (taux de réponse, croissance)
- Funnel (goulets d'étranglement)
- Templates (heatmap, mots gagnants)
- Agent (taux d'échec)

4. ConversionIntelligence - Funnel de Conversion

Fichier : components/bi/ConversionIntelligence.tsx

Rôle : Visualise le parcours prospect → client avec un diagramme Sankey et des métriques de funnel.

Visualisations :

a) Diagramme Sankey

- Représente le flux entre 6 étapes : Identifiés → Connectés → Contactés → Réponses → Intéressés → Convertis
- Épaisseur des liens = volume de flux
- Couleurs des nœuds = étape du funnel
- Labels positionnés à gauche pour éviter le chevauchement

Configuration Recharts :

```
<Sankey
  data={sankey}
  nodePadding={35} // Espacement vertical entre nœuds
  nodeWidth={5}    // Largeur des barres (fine pour éviter chevauchement)
  linkCurvature={0.6} // Incurvation des liens
  margin={{ left: 70, right: 40 }} // Marges optimisées
/>
```

b) Funnel détaillé avec drop-off

- Barres horizontales animées pour chaque étape

- Badge de conversion vs étape précédente
- Indicateur de pertes (drop-off) en rouge

c) Cycle time moyen

- Graphique à barres horizontales
- Durée moyenne (en jours) pour passer d'une étape à l'autre

d) Cohortes hebdomadaires

- Tableau heatmap montrant l'évolution des cohortes
- Colonnes : Total, Connectés, Contactés, Répondu, Convertis
- Intensité de couleur = volume relatif

API endpoint : `/api/bi/conversion?range={jours}`

5. TemplateLab - A/B Testing de Templates

Fichier : `components/bi/TemplateLab.tsx`

Rôle : Analyse la performance des templates de messages pour optimiser les taux de réponse.

Visualisations :

a) Scatter Plot (Usage × Conversion)

- X-axis : Volume d'utilisation
- Y-axis : Taux de conversion (%)
- Taille du point : Volume d'utilisation
- Permet d'identifier les templates "hidden gems" (peu utilisés mais haute conversion)

b) Mots gagnants (Word Cloud)

- Analyse le vocabulaire des templates performants vs échecs
- Score calculé : (occurrences dans templates gagnants) - (occurrences dans templates perdants)
- Taille de police = score relatif
- Couleur = intensité du score

c) Heatmap jour × heure

- Matrice 7 jours × 14 heures (7h-20h)
- Chaque cellule = taux de réponse pour ce créneau
- Couleur verte = taux élevé, gris = pas de données
- Tooltip avec détails (envoyés, réponses, taux)

d) Longueur du message vs taux de réponse

- Line chart montrant la corrélation
- Buckets : 0-100, 100-200, 200-300, 300-500, 500-800, 800+ caractères
- Permet d'identifier la longueur optimale

e) Top 5 / Flop 5 Templates

- Classement des meilleurs et pires templates
- Affiche le volume d'utilisation et le taux de conversion

API endpoint : `/api/bi/templates?range={jours}`

6. ProspectMap - Intelligence Géographique et ICP

Fichier : `components/bi/ProspectMap.tsx`

Rôle : Analyse la distribution géographique, sectorielle et la qualité ICP des prospects.

Visualisations :

a) Top localisations

- Liste des 12 villes/régions les plus représentées
- Barre de progression = densité relative
- Badge de taux de conversion par localisation
- Score ICP moyen par localisation

b) Treemap des industries

- Visualisation hiérarchique des secteurs
- Surface = volume de prospects
- Couleur = industrie
- Tooltip avec détails (score moyen, conversions)

c) Distribution des scores ICP

- Histogramme des scores par bucket (0-20, 20-40, 40-60, 60-80, 80-100)
- Couleur dégradée du rouge (faible) au vert (élevé)

d) Quadrant ICP (Score × Engagement)

- Scatter plot avec 4 quadrants :
 - **Champions** : Score élevé + Engagement élevé
 - **Hot** : Score élevé + Engagement faible (prioriser)
 - **Warm** : Score faible + Engagement élevé (nurture)
 - **Cold** : Score faible + Engagement faible (ignore)
- Couleur des points = statut actuel du prospect
- Labels de quadrant en overlay

API endpoint : `/api/bi/geo?range={jours}`

7. AgentAnalytics et Forecast

Fichiers : `components/bi/AgentAnalytics.tsx` , `components/bi/Forecast.tsx`

Rôle :

- **AgentAnalytics** : Monitoring des actions automatisées de l'agent (succès/échec, types d'actions)
 - **Forecast** : Prédictions basées sur les tendances historiques (non implémenté dans cette version)
-

Implémentation Technique

1. Création des API Routes Next.js

Chaque module BI correspond à une API route dans `app/api/bi/` .

Exemple : `/api/bi/kpi/route.ts`

```
import { NextResponse } from "next/server";
import { pool } from "@/lib/db";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const range = searchParams.get("range") || "30";

  // Requête SQL avec agrégations
  const result = await pool.query(`
    WITH current AS (
      SELECT
        COUNT(*) as prospects,
```

```

        COUNT(*) FILTER (WHERE status = 'converted') as conversions,
        ...
    FROM prospects
    WHERE created_at > NOW() - INTERVAL '${range} days'
),
previous AS (
    SELECT ... WHERE created_at > NOW() - INTERVAL '${range * 2} days'
    AND created_at <= NOW() - INTERVAL '${range} days'
)
SELECT
    current.prospects,
    (current.prospects - previous.prospects) * 100.0 / NULLIF(previous.prospects, 0) as delta,
    ...
FROM current, previous
`);

return NextResponse.json({ success: true, kpis: result.rows[0] });
}

```

Points clés :

- Utilisation de CTEs (Common Table Expressions) pour la comparaison période courante/précédente
- `FILTER (WHERE ...)` pour les agrégations conditionnelles (PostgreSQL)
- Gestion des divisions par zéro avec `NULLIF`
- Calcul du delta en pourcentage

2. Composants React avec Data Fetching

Pattern utilisé : `useEffect` + `useState` + `fetch`

```

export default function KPIHero({ range }: { range: BIRange }) {
    const [data, setData] = useState<KPIData | null>(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        let cancel = false;
        setLoading(true);
        fetch(`/api/bi/kpi?range=${range}`)
            .then(r => r.json())
            .then(j => {
                if (!cancel && j.success) setData(j.kpis);
            })
            .catch(() => {})
            .finally(() => !cancel && setLoading(false));
        return () => { cancel = true; }; // Cleanup
    }, [range]);

    // Rendu conditionnel avec skeleton
    return (
        <div>
            {loading ? <Skeleton /> : <KPICards data={data} />}
        </div>
    );
}

```

Points clés :

- Pattern cancellation pour éviter les race conditions
- Propagation du `range` comme dépendance du `useEffect`
- Skeletons de chargement pour meilleure UX

3. Visualisations avec Recharts

Exemple : Scatter Plot dans TemplateLab

```
<ResponsiveContainer width="100%" height="100%">
  <ScatterChart margin={{ top: 10, right: 20, left: 0, bottom: 10 }}>
    <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" />
    <XAxis type="number" dataKey="usage" name="Utilisations" />
    <YAxis type="number" dataKey="conversion" name="Conversion" unit="%" />
    <ZAxis type="number" dataKey="usage" range=[[60, 400]] />
    <Tooltip content={({ active, payload }) => <CustomTooltip payload={payload} /> } />
    <Scatter data={templates} fill="#a855f7" fillOpacity={0.7} />
  </ScatterChart>
</ResponsiveContainer>
```

Points clés :

- `ResponsiveContainer` pour adaptation responsive
- `ZAxis` pour la taille des points (scatter 3D)
- Tooltip personnalisé pour afficher plus de détails

4. Animations avec Framer Motion

```
<motion.div
  initial={{ opacity: 0, y: 10 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4, delay: i * 0.05 }}
  className="card"
>
  { /* Contenu */ }
</motion.div>
```

Points clés :

- Animation d'entrée en cascade (delay basé sur l'index)
- Transitions fluides pour une UX premium
- `initial` / `animate` pattern pour les animations simples

5. Types TypeScript Partagés

Fichier : `components/bi/biTypes.ts`

```
export type BIRange = 7 | 30 | 90;

export const RANGE_OPTIONS = [
  { value: 7, label: "7j" },
  { value: 30, label: "30j" },
  { value: 90, label: "90j" }
] as const;
```



```

export function fmt(n: number): string {
  return n >= 1000 ? (n / 1000).toFixed(1) + "k" : n.toString();
}

export function downloadCSV(filename: string, data: any[]) {
  const headers = Object.keys(data[0]).join(",");
  const rows = data.map((row) => Object.values(row).join(","));
  const csv = [headers, ...rows].join("\n");
  const blob = new Blob([csv], { type: "text/csv" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  a.click();
}

```

Base de Données et Modélisation

Schéma de la Base de Données

Tables principales utilisées par le BI :

```

-- Prospects
CREATE TABLE prospects (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  company TEXT,
  role TEXT,
  location TEXT,
  linkedin_url TEXT UNIQUE,
  status TEXT DEFAULT 'identified', -- identified, connected, contacted, responded, interested, converted
  score INTEGER DEFAULT 0, -- Score ICP 0-100
  created_at TIMESTAMP DEFAULT NOW()
);

-- Messages
CREATE TABLE messages (
  id SERIAL PRIMARY KEY,
  recipient_name TEXT,
  campaign_id INTEGER REFERENCES campaigns(id),
  template_id INTEGER REFERENCES templates(id),
  status TEXT DEFAULT 'pending', -- pending, sent, replied, read
  content TEXT,
  length INTEGER, -- Longueur en caractères
  created_at TIMESTAMP DEFAULT NOW()
);

-- Templates
CREATE TABLE templates (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  tag TEXT,
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

```

```

-- Campaigns
CREATE TABLE campaigns (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  status TEXT DEFAULT 'draft', -- draft, active, paused, completed
  target TEXT,
  location TEXT,
  contacted INTEGER DEFAULT 0,
  replied INTEGER DEFAULT 0,
  converted INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Agent tool steps (actions de l'agent)
CREATE TABLE agent_tool_steps (
  id SERIAL PRIMARY KEY,
  tool_name TEXT,
  status TEXT, -- success, error
  created_at TIMESTAMP DEFAULT NOW()
);

-- LinkedIn actions queue
CREATE TABLE linkedin_actions_queue (
  id SERIAL PRIMARY KEY,
  action_type TEXT,
  status TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

```

Requêtes SQL Complexes

Exemple : Calcul du funnel de conversion

```

WITH funnel_steps AS (
  SELECT
    'Identifiés' as stage, 'identified' as key, COUNT(*) as value, '#94a3b8' as color
  FROM prospects WHERE created_at > NOW() - INTERVAL '${range}' days'
  UNION ALL
  SELECT 'Connectés', 'connected', COUNT(*), '#60a5fa'
  FROM prospects WHERE status = 'connected' AND created_at > NOW() - INTERVAL '${range}' days'
  UNION ALL
  SELECT 'Contactés', 'contacted', COUNT(*), '#3b82f6'
  FROM prospects WHERE status = 'contacted' AND created_at > NOW() - INTERVAL '${range}' days'
  UNION ALL
  SELECT 'Réponses', 'responded', COUNT(*), '#8b5cf6'
  FROM prospects WHERE status = 'responded' AND created_at > NOW() - INTERVAL '${range}' days'
  UNION ALL
  SELECT 'Intéressés', 'interested', COUNT(*), '#a855f7'
  FROM prospects WHERE status = 'interested' AND created_at > NOW() - INTERVAL '${range}' days'
  UNION ALL
  SELECT 'Convertis', 'converted', COUNT(*), '#10b981'
  FROM prospects WHERE status = 'converted' AND created_at > NOW() - INTERVAL '${range}' days'
),
ordered AS (
  SELECT *, ROW_NUMBER() OVER (ORDER BY

```

```

CASE key
  WHEN 'identified' THEN 1
  WHEN 'connected' THEN 2
  WHEN 'contacted' THEN 3
  WHEN 'responded' THEN 4
  WHEN 'interested' THEN 5
  WHEN 'converted' THEN 6
END
) as rn
FROM funnel_steps
)
SELECT
  stage, key, value, color,
  LAG(value) OVER (ORDER BY rn) as prev_value,
  CASE
    WHEN LAG(value) OVER (ORDER BY rn) > 0
    THEN ROUND(value * 100.0 / LAG(value) OVER (ORDER BY rn), 1)
    ELSE 0
  END as conversionFromPrev,
  CASE
    WHEN LAG(value) OVER (ORDER BY rn) > 0
    THEN LAG(value) OVER (ORDER BY rn) - value
    ELSE 0
  END as dropOff
FROM ordered
ORDER BY rn;

```

Points clés :

- `UNION ALL` pour agréger les étapes du funnel
- `ROW_NUMBER()` pour ordonner les étapes
- `LAG()` pour accéder à la valeur de l'étape précédente
- Calcul de conversion et drop-off avec fenêtres glissantes

Défis Techniques et Solutions

Défi 1 : Performance des Requêtes SQL

Problème : Les requêtes d'agrégation sur de grandes tables pouvaient être lentes.

Solution :

- Utilisation d'index sur les colonnes fréquemment filtrées (`created_at` , `status`)
- Limitation des résultats avec `LIMIT` pour les visualisations (ex: top 12 localisations)
- Caching côté client avec `useEffect` et dépendances intelligentes

Défi 2 : Chevauchement des Labels dans le Sankey

Problème : Les labels des nœuds Sankey se chevauchaient horizontalement.

Solution :

- Réduction de `nodeWidth` de 14 à 5 pixels
- Augmentation de `nodePadding` de 20 à 35 pixels
- Positionnement de tous les labels à gauche (`isLeft = index < 0`)
- Ajustement des marges (`left: 70`, `right: 40`) pour étirer le diagramme
- Réduction de la taille de police (9px pour le nom, 8px pour la valeur)

Défi 3 : Données Insuffisantes pour les Visualisations

Problème : Les visualisations affichaient "Pas de données" avec le jeu de données initial.

Solution :

- Création de scripts de peuplement (`seed-*.js`) :
 - `seed-bi-data.js` : Données de base
 - `seed-extra-data.js` : 80 prospects, 150 messages
 - `seed-message-lengths.js` : Messages avec longueurs variées
 - `seed-prospect-map.js` : 200 prospects avec géographie et industries
- Utilisation de distributions réalistes (score ICP, statuts, timestamps)

Défi 4 : Gestion du Filtre Temporel Global

Problème : Le filtre de période devait être synchronisé entre tous les modules.

Solution :

- État `range` géré dans `BIShell.tsx` (composant parent)
- Propagation via props à tous les composants enfants
- Re-fetch automatique des données quand `range` change (dépendance `useEffect`)

Défi 5 : Suppression de la Fonctionnalité ROI

Problème : L'utilisateur a demandé la suppression de la fonctionnalité "ROI Agent IA" qui causait des erreurs.

Solution :

- Suppression de la bande ROI dans `KPIHero.tsx` (JSX + imports)
- Retrait des propriétés `roi` et `conversionRate` de l'interface `KPIData`
- Suppression des calculs ROI dans `/api/bi/kpi/route.ts`
- Nettoyage des références dans `AIInsights.tsx` (section "Agent ROI")

Conclusion

Le module Business Intelligence représente une valeur ajoutée significative pour l'application LinkedIn Agent Automation. Il transforme les données brutes de prospection en insights actionnables grâce à :

1. **Architecture modulaire** : Chaque module BI est indépendant et réutilisable
2. **Visualisations riches** : Utilisation de Recharts pour des graphiques professionnels (Sankey, Treemap, Scatter, Heatmap)
3. **Performance optimisée** : Requêtes SQL avec agrégations côté serveur
4. **UX premium** : Animations fluides, skeletons de chargement, design responsive
5. **Extensibilité** : Structure permettant d'ajouter facilement de nouveaux KPIs ou visualisations

Ce dashboard BI permet aux utilisateurs de prendre des décisions data-driven pour optimiser leur prospection LinkedIn, identifiant les templates performants, les créneaux horaires optimaux, et les prospects à prioriser selon leur score ICP.

Annexes

A. Liste des Endpoints API

Endpoint	Méthode	Paramètres	Retour
<code>/api/bi/kpi</code>	GET	<code>range (7, 30, 90)</code>	KPIs principaux avec deltas
<code>/api/bi/conversion</code>	GET	<code>range</code>	Funnel, cycle time, cohortes, Sankey
<code>/api/bi/templates</code>	GET	<code>range</code>	Templates, heatmap, mots gagnants

/api/bi/geo	GET	range	Géographie, industries, ICP
/api/bi/agent	GET	range	Analytics agent IA

B. Bibliothèques Utilisées

- **Recharts** : Graphiques (v2.12+)
- **Framer Motion** : Animations (v11+)
- **Lucide React** : Icônes
- **Tailwind CSS** : Styling utilitaire
- **shadcn/ui** : Composants UI (Card, Badge, Button)
- **pg** : Client PostgreSQL (v8.11+)

C. Références

- Documentation Next.js : <https://nextjs.org/docs>
- Documentation Recharts : <https://recharts.org>
- Documentation PostgreSQL : <https://www.postgresql.org/docs>

Fin du document